

Actividad Práctica II

Automatización de modelado, evaluación y despliegue del proyecto de ciencia de datos

Nombre completo	José Escorcía-Gutierrez
Repositorio GitHub	https://github.com/josescorcia/13MBID-AP1-Jose-Escorcía.git
GitHub Project	https://github.com/users/josescorcia/projects/1/views/1
Proyecto	Predicción de mora en créditos a partir de datos de créditos y productos de tarjeta

Resumen ejecutivo

La Actividad Práctica II da continuidad al trabajo desarrollado en la AP1. En esta entrega se automatizan las fases de modelado, evaluación y despliegue del proyecto, incorporando prácticas de MLOps orientadas al registro de experimentación, versionado del modelo, pruebas de regresión y publicación de productos para usuarios finales.

El escenario se mantiene sobre el proyecto de créditos trabajado en la AP1. La variable objetivo es `falta_pago`, que permite estimar la posibilidad de que una cliente presente mora. A partir del dataset integrado generado en la fase de preparación, se comparan cuatro modelos de clasificación y se selecciona el de mejor desempeño según F1 macro sobre el conjunto de prueba.

[A] Comprensión del negocio y planificación del Sprint 2

Objetivo de la organización

La entidad financiera busca mejorar la toma de decisiones en la gestión del riesgo de crédito. Para ello, se propone un producto de datos que permita estimar la posibilidad de mora de un cliente a partir de información histórica de créditos y productos financieros asociados.

Objetivo técnico de la AP2

- Automatizar la comparación de modelos de clasificación para el escenario de mora.
- Registrar la experimentación mediante MLflow.
- Exportar el mejor modelo entrenado y su metadata para uso posterior.
- Generar controles automatizados de evaluación mediante PyTest.
- Extender el pipeline DVC con etapas de modelado y evaluación.
- Construir una API con FastAPI y una interfaz web con Streamlit para el uso del modelo.

Sprint Backlog de la segunda iteración

ID	Historia de usuario	Descripción
HU09	Registrar experimentación de modelos con MLflow	Implementar el seguimiento de parámetros, métricas y artefactos generados durante el entrenamiento.
HU10	Comparar modelos de clasificación	Evaluar Regresión Logística, LinearSVC, KNN y Árbol de Decisión con validación cruzada y conjunto de prueba.
HU11	Exportar modelo y metadata	Guardar el modelo seleccionado en formato pkl y generar un archivo JSON con columnas esperadas y ejemplo de entrada.
HU12	Automatizar evaluación del modelo	Crear reporte de evaluación y tests de regresión para validar desempeño mínimo.
HU13	Actualizar pipeline DVC	Agregar etapas de modelado y evaluación al flujo reproducible del proyecto.
HU14	Implementar API de predicción	Crear endpoints de salud, información del modelo y predicción.
HU15	Implementar app web de consulta	Crear una interfaz Streamlit para enviar registros a la API.

[B] Continuidad de comprensión y preparación de datos

La AP2 reutiliza la salida preparada de la AP1: data/processed/datos_integrados.csv. Este dataset contiene los atributos integrados de créditos y tarjetas, así como variables derivadas generadas durante la preparación de datos.

Dataset de entrada	data/processed/datos_integrados.csv
Filas	8899
Columnas	23
Variables predictoras usadas en modelado	20
Variable objetivo	falta_pago
Tasa de clase positiva	0.1759

El script de entrenamiento elimina del conjunto de variables predictoras las columnas id_cliente, falta_pago y falta_pago_bin, evitando utilizar identificadores o duplicados de la variable objetivo como atributos de entrada.

[D] Modelado

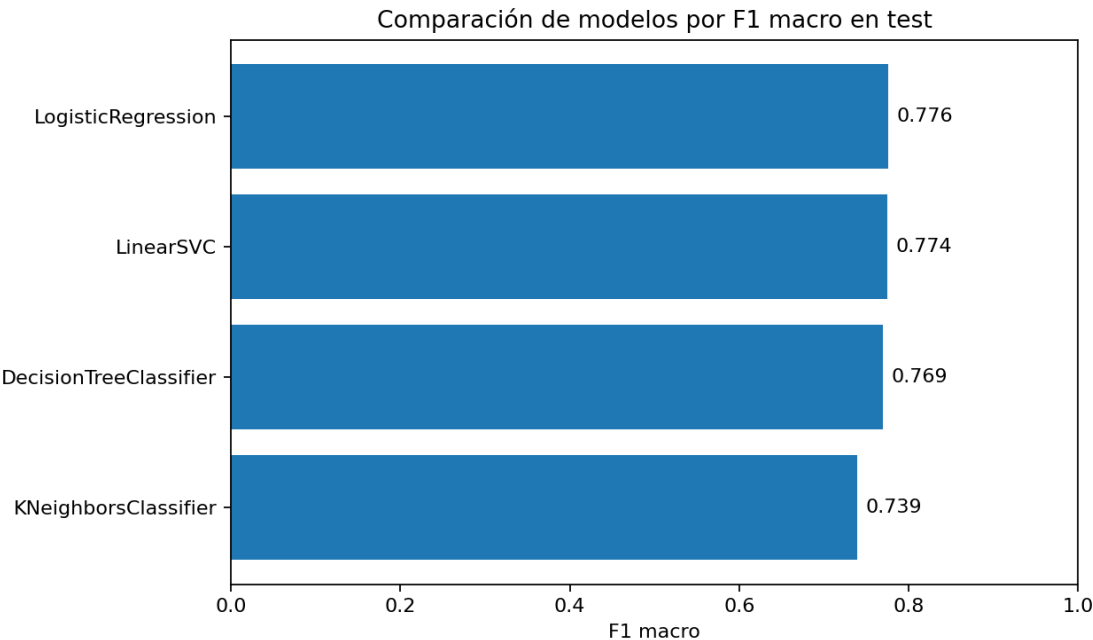
Selección de técnicas

Se definieron cuatro técnicas de clasificación para comparar su desempeño bajo un mismo flujo de preprocesamiento: Regresión Logística, LinearSVC, KNeighborsClassifier y DecisionTreeClassifier.

- Variables numéricas: estandarización mediante StandardScaler.
- Variables categóricas: codificación OneHotEncoder con manejo de categorías no vistas.
- División entrenamiento/prueba: 80% y 20% con estratificación.
- Balanceo: submuestreo determinístico de la clase mayoritaria aplicado únicamente sobre entrenamiento.
- Validación cruzada: 5 folds estratificados sobre el entrenamiento balanceado.
- Métrica principal: F1 macro, por tratarse de un problema con desbalance de clases.

Comparación de modelos

Modelo	CV F1 macro	CV Recall macro	Test accuracy	Test F1 macro	Test Recall macro	Test Precision macro
LogisticRegression	0.8598	0.8598	0.8438	0.7759	0.8399	0.7479
LinearSVC	0.8602	0.8602	0.8421	0.7745	0.8401	0.7465
DecisionTreeClassifier	0.8813	0.8814	0.8281	0.7694	0.8643	0.7428
KNeighborsClassifier	0.8297	0.8298	0.8096	0.7391	0.8153	0.7158



Modelo seleccionado

Modelo seleccionado	LogisticRegression
Accuracy en test	0.8438
F1 macro en test	0.7759
Recall macro en test	0.8399
Precision macro en test	0.7479
Modelo exportado	models/modelo_mora.pkl
Metadata exportada	models/modelo_metadata.json

En la ejecución base, el mejor desempeño según F1 macro en test lo obtuvo LogisticRegression. El resultado se mantiene trazable mediante el archivo reports/metricas_modelo.json y el reporte reports/modelado.md.

```
Modelo seleccionado: LogisticRegression
Métricas exportadas en: C:\Users\Administrador\Desktop\ap2_13mbid_entregable\reports\metricas_modelo.json
Modelo exportado en: C:\Users\Administrador\Desktop\ap2_13mbid_entregable\models\modelo_mora.pkl
```

Registro de experimentación con MLflow

El script src/entrenar_modelos.py registra los experimentos en MLflow cuando la librería está instalada. Para cada modelo se registran parámetros del entrenamiento, cantidad de muestras, método de balanceo, métricas de validación cruzada y métricas de test. Adicionalmente, se registra una corrida final para el modelo seleccionado.

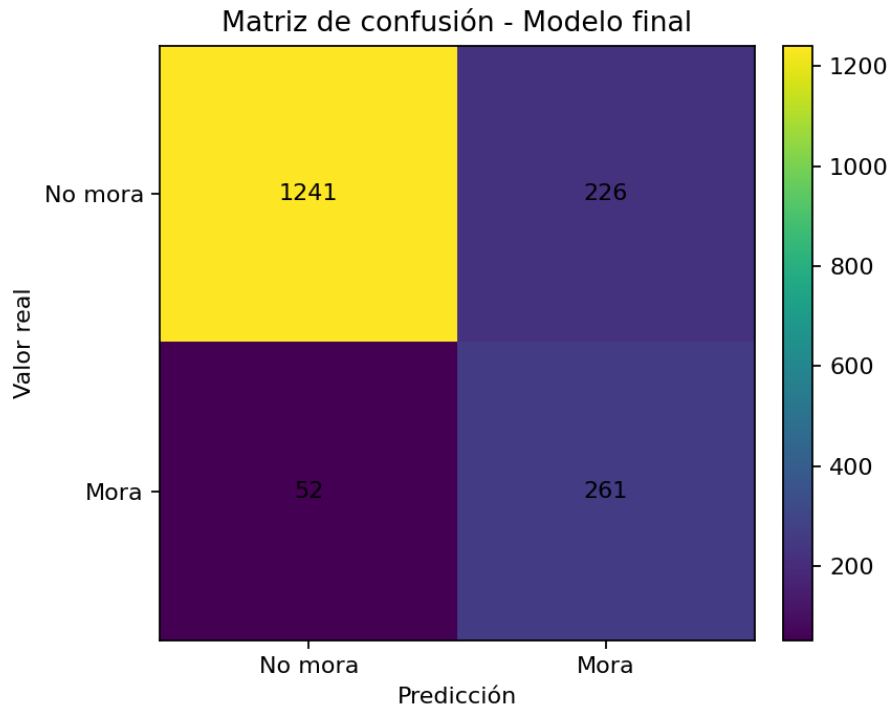
```
Experimento: 1 AP2_Modelado_Mora_Creditos
Cantidad de runs: 5
run_id      tags.mlflow.runName  metrics.test_accuracy  metrics.test_f1_macro  metrics.test_recall_macro
b21d2874010042469476afa5dfcdble0  modelo_final_LogisticRegression  0.843820  0.775888  0.839905
938d44c8f4c6487a966e2ddc5d52838e  DecisionTreeClassifier  0.828090  0.769398  0.864290
c1a10586bcc34151a5e9022ebcd9102e  KNeighborsClassifier  0.809551  0.739093  0.815344
cf6278e9d61e4d44b5816a7488d73915  LinearSVC  0.842135  0.774468  0.840139
d0a25378c3884e3085764292f75bfa12  LogisticRegression  0.843820  0.775888  0.839905
```

[E] Evaluación

Evaluación del resultado

La evaluación se automatizó mediante el script `src/evaluar_modelo.py`, que genera un reporte en Markdown a partir de las métricas exportadas durante el entrenamiento. El objetivo es contar con un control reproducible para verificar que el modelo mantiene un rendimiento mínimo ante nuevas ejecuciones.

Real 0 / Predicción 0	1241
Real 0 / Predicción 1	226
Real 1 / Predicción 0	52
Real 1 / Predicción 1	261



Pruebas automatizadas

- `test_metricas_modelo_existen_y_superan_umbral`: valida que existan métricas y que F1 macro y recall macro superen el umbral mínimo definido.
- `test_modelo_exportado_puede_generar_predicciones`: valida que el modelo exportado pueda cargarse y generar una predicción para un registro de ejemplo.

```
(.venv) C:\Users\Administrador\Desktop\ap2_13mbid_entregable>python -m pytest -q
..... [100%]

----- warnings summary -----
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
tests/test_modelo.py::test_modelo_exportado_puede_generar_predicciones
C:\Users\Administrador\Desktop\ap2_13mbid_entregable\.venv\Lib\site-packages\joblib\numpy_pickle.py:287: DeprecationWarning: Setting the shape on a NumPy array has been deprecated in NumPy 2.5.
As an alternative, you can create a new view using np.reshape (with copy=False if needed).
array.shape = self.shape
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
8 passed, 7 warnings in 1.56s
```

Pipeline DVC actualizado

El archivo `dvc.yaml` fue extendido con las etapas `modelado` y `evaluacion_modelo`. Esto permite ejecutar en secuencia la preparación de datos, el entrenamiento, la generación de reportes y la evaluación automatizada.

- visualizacion: genera gráficos y resumen de datos.
- validacion_calidad: ejecuta validaciones de calidad y exporta reportes.
- preparacion_datos: construye el dataset integrado final.
- modelado: compara modelos, registra experimentación y exporta el modelo seleccionado.
- evaluacion_modelo: genera el reporte de evaluación del modelo seleccionado.

```
Running stage 'visualizacion':
> python src/generar_visualizaciones.py
Visualizaciones exportadas en: G:\Otros ordenadores\Mi portátil\Maestria\Metodologías de gestión y diseño proyectos BD\Entregables act1\api_13mbid_entregable\reports\figures
Resumen exportado en: G:\Otros ordenadores\Mi portátil\Maestria\Metodologías de gestión y diseño proyectos BD\Entregables act1\api_13mbid_entregable\reports\resumen_datos.md

Running stage 'validacion_calidad':
> python src/validar_calidad.py
Validación finalizada.
Updating lock file 'dvc.lock'

Running stage 'preparacion_datos':
> python src/preparar_datos.py
Dataset preparado exportado en: G:\Otros ordenadores\Mi portátil\Maestria\Metodologías de gestión y diseño proyectos BD\Entregables act1\api_13mbid_entregable\data\processed\datos_integrados.csv
Filas: 8899 | Columnas: 23
Updating lock file 'dvc.lock'
Use 'dvc push' to send your updates to remote storage.

(.venv) G:\Otros ordenadores\Mi portátil\Maestria\Metodologías de gestión y diseño proyectos BD\Entregables act1\api_13mbid_entregable>
```

[F] Despliegue / Implementación

Plan de implementación

Se implementaron dos productos de despliegue para facilitar el uso del MVP por usuarios finales: una API con FastAPI y una aplicación web con Streamlit. La API expone el modelo entrenado a través de endpoints de consulta y predicción, mientras que Streamlit ofrece una interfaz para cargar los valores del cliente/crédito y enviar la solicitud a la API.

API	app/api.py
Aplicación web	app/streamlit_app.py
Guía de despliegue	deployment/README_DESPLIEGUE.md
Configuración Render	deployment/render_api.yaml

API FastAPI

- GET /: endpoint raíz del servicio.
- GET /health: verificación de disponibilidad del modelo y metadata.
- GET /model-info: descripción del modelo y columnas esperadas.
- POST /predict: generación de predicción para un registro enviado por el usuario.

API AP2 13MBID - Predicción de mora

2.0.0 OAS 3.1

/openapi.json

Servicio para utilizar el modelo de predicción de falta de pago del proyecto de créditos.

default

GET	/	Root	✓
GET	/health	Health	✓
GET	/model-info	Model Info	✓
POST	/predict	Predict	✓

Schemas

HTTPValidationError > Expand all object

PredictionRequest > Expand all object

Aplicación Streamlit

La aplicación Streamlit permite ingresar los datos de entrada del modelo, configurar la URL de la API y visualizar el resultado de predicción en términos interpretables: mora o no mora.

AP2 13MBID - Herramienta de predicción de mora

Aplicación web de consulta para usuarios finales. Permite enviar un registro a la API del modelo entrenado.

Datos del cliente/crédito

importe_solicitado	35000,00	duracion_credito	3,00
situacion_vivienda	ALQUILER	ingresos	59000,00
objetivo_credito	PERSONAL	pct_ingreso	0,59
tasa_interes	16,02	antiguedad_cliente	36,00
estado_cliente	ACTIVO	gastos_ult_12m	1088,00
genero	M	limite_credito_tc	4010,00
nivel_educativo	UNIVERSITARIO_COMPLETO	operaciones_ult_12m	24,00
personas_a_cargo	2,00	estado_civil_N	C
estado_credito_N	C	antiguedad_empleado_N	sin_categoria
edad_N	menor_25	ratio_gasto_limite_tc	0,27

Generar predicción

personas_a_cargo

2,00

-

+

estado_civil_N

C

estado_credito_N

C

antiguedad_empleado_N

sin_categoria

edad_N

menor_25

ratio_gasto_limite_tc

0,27

-

+

Generar predicción

Resultado: Mora

```

{
  "prediction": 1,
  "prediction_label": "y",
  "interpretable_result": "Mora"
}

```

Ejemplo de registro

importe_solicitado

duracion_credito

situacion_vivienda

ingresos

objetivo_credito

pct_ingreso

tasa_interes

antiguedad_cliente

estado_cliente

gastos_ult_12m

genero

limite_credito_tc

nivel_educativo

operaciones_ult_12m

personas_a_cargo

estado_civil_N

estado_credito_N

Informe final y revisión del proyecto

La segunda iteración permitió completar la transición del proyecto desde un flujo de preparación de datos hacia un producto de datos operativo. Se incorporó registro de experimentos, evaluación reproducible, exportación del modelo y componentes de despliegue orientados a usuarios finales.

Elementos desarrollados en las dos iteraciones

- Iteración 1: estructura del repositorio, versionado de datos, visualizaciones, validación de calidad, preparación de datos y pipeline DVC inicial.
- Iteración 2: modelado comparativo, registro con MLflow, exportación de modelo, evaluación con pruebas de regresión, API FastAPI y aplicación Streamlit.